INFO212 Systemutvikling · Autumn 2026 · Obligatory assignment 4

Building Ekte Mat Online with Claude Code

From your Claude Cowork lab exercise (https://mitt.uib.no/courses/60898/files/folder/Lab%20exercises?preview=8086954) requirements to a working Flask application, built with Claude Code and GitHub

Group work	2–4 students, one submission per group. Every member commits to the GitHub repository.
Builds on	Lab exercise (Claude Cowork): the data pack, your backlog and your models.
Hand in	One PDF report with a link to your GitHub repository, on Mitt UiB. Deadline: see Mitt UiB.
Assessment	Approved / not yet approved. The assignment is one of the compulsory course requirements.

1 The assignment

In Lab 7 you were the analysts for **Ekte Mat AS**, the fictional Bergen catering company that wants to replace ordering by e-mail, phone and Excel. Now your group is the development team. Build a working first version of **Ekte Mat Online**, a customer website and an admin panel, with Claude Code as your coding agent and GitHub as your shared workspace.

Start from the requirements in the data pack and your Lab exercise on Claude Cowork backlog and models. You may go beyond them by adding features or resolving the conflicts you found, as long as you can name the stakeholder need each feature serves. The product matters, but so does the process: you should learn to steer an AI agent through a realistic project (plan, build, test, review) and to judge whether its work is right.

2 Functional requirements (minimum)

Admin panel (Ekte Mat staff)	Customer site (the published shop)
A1 Food items. Add, edit and remove the food Ekte Mat caters: name, description, price in NOK, minimum order quantity, event categories, allergens and dietary labels, and whether the item is published.	**C1 Menu by event.** Food items as cards, grouped by kind of event: birthday, confirmation, wedding, conference, Christmas party (julebord) and so on. Each card shows picture, price, minimum order and allergens.
A2 Images. Upload a picture for each item, or write an instruction that an AI agent or image service uses to create or modify the picture. The admin previews the result before it is saved.	**C2 Shopping cart.** Add, change and remove items, with an estimated total cost. Quantities below an item's minimum order are not accepted.
A3 Orders. A list of all customer orders that can be filtered by status and event date, and a page per order to manage it: order status (submitted → approved → in preparation → delivered, or rejected/cancelled) and payment status (e.g. not paid, deposit paid, invoiced, paid).	**C3 Order.** Log in (or register) and submit the order: event date, number of guests, delivery address, and allergy and dietary notes per guest group. No payment solution is needed.
A4 Questions. Read and answer customers' questions about their orders.	**C4 My orders.** The customer's own orders with their current status (submitted, approved, delivered, …) and payment status.
	C5 Questions. Ask a question about a specific order and read Ekte Mat's answer.

Going further (optional), with ideas from the data pack: a kitchen view of the day's orders with allergies highlighted (Mateo), a check against the kitchen's daily capacity (Sunniva), company accounts that can copy last week's order (Fjordtek), the origin of the food on every card (Ingrid), or a Norwegian/English interface.

3 Technical requirements

- **Architecture: Model–View–Controller, with Flask as the backend.** Models hold the domain classes and all data access, views are Jinja2 templates or a separate front end that calls a JSON API, and controllers are Flask routes grouped in blueprints (e.g. shop, account, admin). The three layers must be visible in your folder structure.
- **Front end:** HTML, CSS and JavaScript, or any front-end technology you are familiar with (e.g. React or Vue).
- **Data storage:** Neo4j is the storage layer of the finished system (see the box below). For this assignment a temporary store such as SQLite or JSON files is fine, as long as only repository classes in the model layer touch it, so that Neo4j can later replace it without changes to controllers or views.

- **Security basics:** Only customer login pages are fine. While comming code on github – make sure that your project does not include Claude API key.
- **Running it:** synthetic data only. A seed script creates sample food items, an admin and test customers, and the README gives the commands to install, seed, run and test the application.

> **Neo4j and Obligatory assignment 5**
> You do not have to deliver the Neo4j implementation now; it is part of Obligatory assignment 5. If you already implement the storage layer with Neo4j, say so on the front page of your report. Your submission then also counts as Obligatory assignment 5, and you do not have to deliver it separately.

4 Working with Claude Code and GitHub

Use this project to explore Claude Code in depth. Your repository must show at least the following:

- **CLAUDE.md:** the project briefing (what the system is, the MVC layout, how to run and test it, and your team's rules). Keep it short and true.
- **Plan mode:** plan every larger feature before building it, and commit the agreed plan to `docs/plans/`.

5 Report and submission

Hand in one PDF on Mitt UiB that contains:

1. **Front page:** group members, the link to the GitHub repository and the tag of the version to assess (e.g. `oblig4`), and whether Neo4j is included (yes/no). The repository must be public, private and shared with the course staff.
2. **Features:** the name and a one-line description of each feature you built, with screenshots of the different screens: at least the menu, cart, order form, my orders and questions on the customer site, and the food item, image, order list and order pages in the admin panel.
3. **Software design:** the design concepts you used, such as user stories, a use-case diagram, a domain class diagram, a sequence diagram, a state machine for the order status, an MVC architecture sketch or a graph data model. Include the diagrams and say in a sentence or two how each one was used.
4. **Did design help?** A short paragraph on whether your knowledge of software design helped you build the product and steer Claude Code.

To be approved: the application should be executable and provides the functionalities listed in the project description. The code follows MVC, and the report contains parts 1–4.